HabitFlow

Owner B — Analytics & Feature Modules

Product Requirements Document + AI Build Specification

Self-contained implementation brief derived from the master HabitFlow BRD and build specification. Shared requirements apply to both collaborators; ownership boundaries identify the work assigned to this document.

Implementation goal

Build independent analytics, reminders/PWA, and assigned Phase 2 modules against the agreed shared interfaces without replacing the core architecture.

Collaboration rule

Use one shared GitHub repository. Each person works on a feature branch and opens a pull request. Keep main buildable, agree shared interfaces before parallel work, and pull merged changes into the local Antigravity clone.

How to use this document

This document includes the shared BRD requirements needed to understand the product, followed by this collaborator's implementation scope. The master combined PDF remains authoritative. Do not independently implement the other person's module unless both collaborators agree.

Ownership boundary

Assigned owner: Owner B — Analytics & Feature Modules. Shared files such as database schema, global types, authentication, date/time utilities, app layout, and service-worker registration require coordination before edits. Avoid concurrent edits to the same files.

1. Product overview

HabitFlow is a private, customizable hourly habit and behavior tracking web app. Users create questions, answer them across time slots, and understand patterns through a spreadsheet-style Track Sheet and analytics.

Product philosophy: Track honestly → Understand patterns → Measure improvement → Improve gradually. Keep the product fast, private, minimal, professional, data-driven, non-judgmental, and easy to use repeatedly.

2. Critical answer/outcome rules

Raw answer and evaluated outcome are separate. Never assume YES means Positive or NO means Negative. Every option has its own configured Positive, Negative, or Neutral outcome.

Store the selected answer and an outcome snapshot on each entry. The Track Sheet can visualize the outcome; detail views preserve the original answer. Changing option rules must not silently rewrite history. Unanswered is missing data, never a negative result.

3. Shared UX and core flow

Navigation: Dashboard, Track Sheet, Questions, Analytics (Daily/Weekly/Monthly), Insights, Settings. The two screenshots in the master PDF are the visual source of truth. Preserve their recognizable layout, density, spacing, colors, typography, and interactions; do not replace them with a generic dashboard.

Track Sheet rows are questions; columns are time slots; each cell is one question/slot/date. Phase 1 default is 6 AM–9 PM. Selecting a cell opens an answer dialog; save updates the cell without full-page reload. Include date navigation and entry edit/delete.

Question management includes create/edit, category, ordering, activate/pause/archive/delete, configurable answer options, outcome mapping, and optional explanations.

4. Analytics and insights rules

Daily analytics: outcome counts, answered/unanswered slots, completion, and time-of-day summaries. Weekly analytics: seven-day comparison and change over time. Monthly analytics: trends and heatmap. Insights must come from real saved entries and explain the time window/sample size when practical.

Use explicit denominators; do not mislead with one score when questions have different meanings. Correlation does not prove causation. Never fabricate analytics or production sample values.

5. Security and reliability

Authentication: sign-up, login, logout, session persistence, protected routes, strict user-level data isolation. Every server-side read/write must authorize the current user; never trust client-supplied user IDs.

Validate inputs server-side. Keep secrets in environment variables; never expose private keys in browser code. Provide loading, empty, validation, error and success states. Persist data across refresh/navigation; report failed saves honestly.

Use modular components, strict TypeScript where practical, reusable calculations, migrations, and tests. Preserve compatible existing infrastructure and avoid unnecessary paid services.

6. Mobile-first and accessibility

Test widths 320, 360, 375, 390 and 430 CSS px plus tablet/desktop. Avoid accidental page overflow. Keep question labels visible while the hourly grid scrolls horizontally, or use a focused single-question mobile view; do not crush columns.

Use practical 44×44 px touch targets, keyboard access, clear labels/focus, sufficient contrast, responsive charts/tables/dialogs, safe-area spacing, and no essential hover-only controls.

7. Shared technical contract

Preferred stack when compatible: Next.js, React, TypeScript, Tailwind CSS, shadcn/ui, PostgreSQL/Neon, Drizzle ORM, compatible auth such as Better Auth, Recharts, Vercel. Change only for a documented compatibility reason.

Shared entities: auth users/sessions; questions (id, user_id, text, category, answer_type, active, order, timestamps); question_options (id, question_id, text, outcome, explanation, order); hourly_entries (id, user_id, question_id, option/value, date, time slot, outcome_snapshot, timestamps); settings/reminder preferences as needed.

Before parallel development, agree TypeScript types, data-access/query functions, route shapes, timezone/date rules, shared file ownership, environment variables, and migration workflow. Never build against guessed database fields or create a second auth/database architecture.

8. Phases

Phase 1: foundation, auth/authorization, database, Yes/No question management and outcomes, Track Sheet, entry management, dashboard, daily/weekly/monthly analytics, basic insights, responsive polish and reliability.

Phase 2 is planned scope: custom tracking hours/intervals; single-choice, multiple-choice, numeric, currency, duration and rating questions; goals/streaks; CSV/PDF exports; advanced behavioral correlations; optional AI summaries only after basic analytics are reliable. Build extensible foundations now and implement Phase 2 in separate PRs after dependencies are ready.

9. Notifications and PWA

Support interval reminders and specific clock-time reminders, timezone/DST, configured quiet hours/weekdays, duplicate prevention, sensible missed reminders, subscription expiry/recovery, and navigation to Track Sheet. Avoid sensitive habit answers in push payloads by default.

Background reminders require supported service worker/Web Push and server-side scheduling. Browser timers alone cannot guarantee delivery after browser closure. Explain platform/permission limits accurately.

PWA: manifest, app name/icons, standalone display, HTTPS service worker, install action/manual instructions, installed-state handling, safe offline fallback and updates that preserve unsaved entries. Do not indiscriminately cache private authenticated APIs.

10. Definition of done

A feature is not complete just because the UI exists. Verify persistence, authorization, validation, loading/empty/error states, responsive behavior, tests and build/type-check/lint results. Include migrations and document blocked checks.

Inspect repository and Git status first. Work in focused branches; never commit secrets. Each PR explains purpose, files, screenshots for UI changes, tests, migration/environment needs, and unresolved issues.

Assigned implementation scope

Build independent analytics, reminders/PWA, and assigned Phase 2 modules against the agreed shared interfaces without replacing the core architecture.

B1. Analytics and insights

- Build daily/weekly/monthly pages, charts, heatmaps, completion/outcome summaries, question-level trends and evidence-based time-of-day insights from the shared data contract.
- Use typed mock data only for isolated layout work while the contract is being finalized; replace all mock data before completion.
- Keep calculations tested and consistent with shared outcome, missing-data, denominator and timezone rules.

B2. Notifications and reminders

- Build settings UI for interval schedules and clock times. Handle timezone, configured quiet hours/weekdays, duplicates, missed reminders, subscription lifecycle and failure states.
- Coordinate server scheduler/queue and Web Push configuration with Owner A. Client timers alone are not acceptable for background reminders. Document deployment/environment prerequisites.

B3. PWA and safe offline behavior

- Implement manifest, 192×192 and 512×512 icons (maskable where supported), standalone display, service worker, install action/manual instructions, installed-state handling, notification-click routing and safe offline fallback.
- Coordinate service-worker registration with push; do not create competing registrations. Avoid caching private APIs; preserve unsaved work during updates and document platform limitations.

B4. Phase 2 modules

- Goals/streaks: user-defined targets, progress, clear rules for incomplete slots and days.
- CSV/PDF exports: only current user’s selected range/filters; handle empty data, escaping, readable PDF and privacy.
- Advanced correlations: require sufficient/complete data where possible; show time window/sample size and explain correlation is not causation.
- Optional AI summaries only after analytics are validated. Minimize data sent to models, protect sensitive records and separate measured facts from generated interpretation.

B5. Owner B acceptance

- Analytics uses actual saved entries and agreed calculations; no mock production values remain.
- Reminder flows handle denied permission, unsupported platforms, timezone/schedule changes, duplicate prevention and background-delivery limitations.
- PWA installation/offline behavior is safe and explicit. Each Phase 2 module has tests, responsive UI, empty/loading/error states and a focused PR.
- Each PR lists changed files, migrations/environment needs, tests, screenshots for UI changes and unresolved issues.

Implementation process and GitHub workflow

Step	Required process
1. Inspect before editing	Open the repository in Antigravity. Review README, package scripts, Git status and current code. Do not overwrite uncommitted work.
2. Branch per feature	Update main and create a focused branch such as feat/track-sheet, feat/analytics, feat/reminders or feat/pwa.
3. Agree interfaces	Before relying on schema or shared utilities, agree TypeScript types, query functions, API behavior, date/time rules and shared-file ownership.
4. Implement and test	Add tests, run lint/type-check/build, test responsive states and loading/empty/error states. Report tests accurately.
5. Commit and push	Review git diff; exclude secrets/local data; make a descriptive commit and push the feature branch.
6. Pull request	Explain purpose, requirements, files changed, UI screenshots, migrations, environment changes, test results and blockers.
7. Integrate locally	After merge, run git switch main and git pull origin main in Antigravity's terminal. Install changed dependencies and run migrations if needed.
8. Resolve conflicts safely	Commit or stash local work before switching branches. Merge/rebase only using the agreed team workflow; resolve conflicts deliberately and rerun checks.

Common Git commands

```
git clone https://github.com/OWNER/REPOSITORY.git

cd REPOSITORY

git switch main

git pull origin main

git switch -c feat/your-feature

# after implementation and tests

git add .

git commit -m "feat: describe the feature"

git push -u origin feat/your-feature
```

To receive merged work on this laptop: commit or stash your changes, then run git switch main and git pull origin main. GitHub does not sync files into Antigravity automatically; pull updates the local clone.

Acceptance criteria and release gate

- The UI follows the reference-led design and is responsive on mobile, tablet and desktop.
- All reads/writes are authenticated and scoped to the current user server-side.
- Raw answers and outcome snapshots remain separate; unanswered cells are not negative.
- Data persists across refresh/navigation and failed saves are reported.
- Analytics and exports use real data and documented rules.
- Push/PWA limitations are accurate; private data is not exposed through push payloads or unsafe caches.
- Relevant tests, lint, type-check and production build are run and actual results reported.
- No secrets are committed; migrations/environment changes are documented.
- Pull request is focused and does not silently change shared contracts.

Antigravity prompt for this owner

Read the master HabitFlow Combined BRD + AI Build Specification and this Owner B PRD/build specification. Inspect repository, README, shared data/type contract and Git status before editing. My ownership: analytics/insights, notifications/Web Push, PWA/offline behavior, and Phase 2 goals/streaks, CSV/PDF export, advanced correlations and optional AI summaries after basic analytics are validated. Plan files/interfaces/migrations first. Work on focused branches; do not replace scaffold/auth/database or guess schema fields. Coordinate shared-file changes, use real data, test and report changes/results/blockers. Preserve reference UI, protect private data and never claim correlations prove causation.

Phase 2 scope reminder

Phase 2 is planned scope: custom tracking hours/intervals; single-choice, multiple-choice, numeric, currency, duration and rating questions; goals/streaks; CSV/PDF exports; advanced behavioral correlations; and optional AI summaries after basic analytics are reliable. Implement only the assigned slice and coordinate shared dependencies.